

Dispensa · Modulo 8

Il modulo 8 introduce due modi nuovi di fornire dati a un programma Python: la **riga di comando** e i **file**. Fino ad ora i dati venivano inseriti dall'utente tramite `input()` oppure scritti direttamente nel codice. Entrambi questi approcci diventano impraticabili quando i dati sono molti o persistenti: non è realistico digitare mille voti ogni volta che si vuole calcolare una media. I file risolvono questo problema.

1. Argomenti da riga di comando

Come funziona

Quando eseguiamo uno script Python dal terminale, passiamo all'interprete il nome del file da eseguire:

```
python3 script.py
```

È però possibile aggiungere ulteriori valori sulla stessa riga, separati da spazi:

```
python3 script.py ciao 42
```

Questi valori aggiuntivi si chiamano **argomenti da riga di comando**. All'interno del programma sono accessibili tramite la variabile `sys.argv`, che è una lista popolata automaticamente dall'interprete.

Il modulo `sys`

`sys.argv` non è disponibile di default: appartiene al modulo `sys`, che gestisce l'interfaccia tra Python e il sistema operativo. Va importato esplicitamente:

```
import sys
print(sys.argv)
```

Se il programma viene eseguito con:

```
python3 pippo.py ciao 42
```

`sys.argv` conterrà:

```
['pippo.py', 'ciao', '42']
```

Struttura di `sys.argv`

Posizione	Contenuto
0	Nome dello script
1	Primo argomento passato
2	Secondo argomento passato
...	...

La posizione 0 è sempre il nome dello script: i dati utili partono da 1.

Tutto arriva come stringa

Come con `input()`, tutto quello che proviene dalla riga di comando è letto come **stringa**. Se vogliamo trattare un argomento come numero, dobbiamo convertirlo esplicitamente con `int()` o `float()`.

Esempio: somma di due numeri passati da riga di comando.

```
import sys

n1 = int(sys.argv[1])
n2 = int(sys.argv[2])
print(n1 + n2)
```

Esecuzione:

```
python3 somma.py 3 7
10
```

Esempio: massimo di una lista di numeri

Quando gli argomenti sono molti, conviene convertirli tutti in un unico ciclo prima di elaborarli:

```
import sys

# Converte tutti gli argomenti (tranne il nome dello script) in interi
i = 1
while i < len(sys.argv):
    sys.argv[i] = int(sys.argv[i])
    i = i + 1

# Trova il massimo
massimo = sys.argv[1]
i = 1
while i < len(sys.argv):
    if sys.argv[i] > massimo:
        massimo = sys.argv[i]
    i = i + 1

print(massimo)
```

Senza la conversione, il confronto avverrebbe tra stringhe: "11" risulterebbe minore di "9" perché il confronto lessicografico considera il primo carattere ('1' < '9'), non il valore numerico.

Esempio: funzione palindroma applicata a più parole

```
import sys

def is_palindromo(parola):
    i = 0
    j = len(parola) - 1
    palindromo = True
    while i < len(parola) // 2:
        if parola[i] != parola[j]:
            palindromo = False
        i = i + 1
        j = j - 1
    return palindromo

i = 1
while i < len(sys.argv):
    parola = sys.argv[i]
    if is_palindromo(parola):
        print(parola, "sì")
    else:
        print(parola, "no")
    i = i + 1
```

Esecuzione:

```
python3 palindromi.py ciao radar pizza anna
ciao no
radar sì
pizza no
anna sì
```

`is_palindromo` è una **funzione pura**: viene usata come condizione di un `if`, quindi deve restituire un booleano — non può essere `void`.

2. Il filesystem e i percorsi

Struttura ad albero

I file nel computer non sono disposti a caso: sono organizzati in **cartelle** (o directory) annidate le une dentro le altre, formando una struttura ad albero. La radice di quest'albero si chiama `/` (slash) su macOS e Linux, o `C:\` su Windows.

Ogni file ha una **posizione univoca** in quest'albero. Per descrivere questa posizione si usa il **percorso** (path): una stringa che elenca le cartelle da attraversare per arrivare al file, separate da `/`.

Un esempio di albero:

`/`

```

+-- users/
  +-- ludovica/
    +-- informatica-di-base/
      +-- modulo8/
        |   +-- script.py
        |   +-- analisi.py
      +-- dati/
        +-- voti.txt
        +-- nomi.txt

```

Percorso assoluto

Il **percorso assoluto** descrive la posizione del file partendo dalla radice del filesystem. Funziona sempre, indipendentemente da dove ci troviamo:

```

/users/ludovica/informatica-di-base/modulo8/script.py
/users/ludovica/informatica-di-base/dati/voti.txt

```

È l'indirizzo completo del file, come scrivere l'indirizzo postale per intero.

Percorso relativo

Il **percorso relativo** descrive la posizione del file a partire dalla **cartella corrente** — cioè da dove ci troviamo in questo momento. È più breve ma dipende da dove siamo.

Se ci troviamo in `informatica-di-base/`:

```

modulo8/script.py      ← scendi in modulo8, poi prendi script.py
dati/voti.txt          ← scendi in dati, poi prendi voti.txt

```

Se ci troviamo già in `modulo8/`:

```

script.py              ← script.py è qui, non serve specificare altro
../dati/voti.txt        ← risali di un livello (..), poi scendi in dati

```

Il simbolo `..` significa “cartella superiore”. Si può usare più volte:

```

../../altro/file.txt    ← risali due livelli, poi scendi in altro

```

La cartella corrente

In ogni momento il terminale “si trova” in una cartella specifica, chiamata **cartella corrente** o **working directory**. Tutti i percorsi relativi vengono interpretati a partire da essa.

Per sapere dove ci si trova si usa `pwd` (print working directory):

```

$ pwd
/users/ludovica/informatica-di-base

```

Per spostarsi si usa `cd` (change directory):

```
$ cd modulo8
$ pwd
/users/ludovica/informatica-di-base/modulo8
```

Per risalire di un livello:

```
$ cd ..
$ pwd
/users/ludovica/informatica-di-base
```

Comandi bash essenziali

Comando	Significato	Esempio
<code>pwd</code>	Mostra la cartella corrente	<code>pwd</code>
<code>ls</code>	Elenca file e cartelle nella posizione corrente	<code>ls</code>
<code>ls</code>	Elenca il contenuto di un'altra cartella	<code>ls dati/</code>
<code>percorso</code>		
<code>cd nome</code>	Entra nella cartella <code>nome</code>	<code>cd modulo8</code>
<code>cd ..</code>	Risale alla cartella superiore	<code>cd ..</code>
<code>cd ~</code>	Va alla home dell'utente	<code>cd ~</code>
<code>mkdir nome</code>	Crea una nuova cartella	<code>mkdir prova</code>
<code>touch nome</code>	Crea un file vuoto	<code>touch note.txt</code>

Il simbolo `~` (tilde) è una scorciatoia per la cartella home dell'utente. Su Mac si digita con `alt+5`, su Windows con `alt+0126`.

Attenzione agli spazi nei nomi dei file. Bash usa lo spazio come separatore tra argomenti: un nome come `mio file.txt` viene interpretato come due argomenti distinti (`mio` e `file.txt`), causando errori difficili da capire. Conviene usare trattini o underscore al posto degli spazi: `mio-file.txt` oppure `mio_file.txt`.

Dove si trova l'interprete Python

Quando eseguiamo `python3 script.py` dal terminale, l'interprete risolve tutti i percorsi relativi **a partire dalla cartella corrente del terminale**, non dalla cartella in cui si trova lo script. Questo è un dettaglio che causa molti errori.

Esempio concreto: supponiamo di avere questa struttura:

```
informatica-di-base-dati/
+-- script/
|   +-- leggi.py           <- contiene: open('dati/voti.txt', ...)
+-- dati/
    +-- voti.txt
```

Il file `leggi.py` usa il percorso relativo `dati/voti.txt`. Questo percorso viene risolto a partire da dove si trova il terminale, non da dove si trova `leggi.py`.

Da dove lancio	Comando	Funziona?	Perché
Python			
informatica-di-base/	<code>python3/ script/leggi.py</code>	Sì	<code>dati/voti.txt</code> esiste rispetto a questa cartella
script/	<code>python3 leggi.py</code>	No	cerca <code>dati/voti.txt</code> dentro <code>script/</code> , ma non c'è

La regola pratica è: prima di eseguire uno script, verificare con `pwd` di trovarsi nella cartella giusta, cioè quella da cui i percorsi relativi scritti nel codice hanno senso.

3. File in Python

Perché i file

`input()` e `sys.argv` vanno bene per dati semplici — pochi valori inseriti manualmente. Non sono adatti quando i dati sono:

- **numerosi**: non è pratico scrivere a mano mille voti dalla riga di comando;
- **persistenti**: esistono indipendentemente dall'esecuzione del programma, salvati su disco.

Un file è una sequenza di dati salvata su disco con un percorso che la identifica univocamente. Python può leggerla e modificarla.

La funzione `open()`

Per accedere al contenuto di un file occorre prima creare un **file handler**: un oggetto Python che rappresenta il collegamento tra il programma e il file su disco. Lo si crea con la funzione `open()`:

```
f = open(percorso, modalità, encoding=codifica)
```

I tre parametri:

Parametro	Tipo	Scopo
<code>percorso</code>	stringa	Percorso assoluto o relativo del file
<code>modalità</code>	stringa	'r' per leggere, 'w' per scrivere
<code>encoding</code>	stringa	Come interpretare i byte del file (di solito 'utf-8')

Codifiche: ASCII e UTF-8

Sul disco i file non contengono lettere: contengono sequenze di zero e uno. La **codifica** è la tabella che associa sequenze di bit a caratteri.

- **ASCII** è la prima codifica diffusa: prevede 128 caratteri, sufficienti per l'alfabeto inglese, i numeri e i simboli base. Non include le lettere accentate di lingue come l'italiano.
- **UTF-8** è la codifica oggi dominante. Contiene tutti i caratteri ASCII (con la stessa rappresentazione) e aggiunge il supporto per lettere accentate, caratteri di qualsiasi lingua, emoji e molto altro. Usa una lunghezza variabile: i caratteri rari occupano più byte.

Se si apre un file con la codifica sbagliata, i caratteri non-ASCII risultano illeggibili o causano un errore. La regola pratica è usare sempre `encoding='utf-8'`; se il file proviene da una fonte esterna (ad esempio Excel, che di default non usa UTF-8) potrebbe essere necessario adattarsi.

Il file handler è un iterabile

Il file handler non è una lista: non supporta l'accesso diretto per indice (`f[120]` non funziona). È però un **iterabile**: può essere usato all'interno di un `for`, che restituisce una riga alla volta. Il file va letto dall'inizio alla fine, in ordine.

Chiudere il file: il costrutto `with`

Ogni file handler aperto occupa risorse di sistema. Il sistema operativo impone un limite al numero di file aperti contemporaneamente; inoltre, le operazioni di scrittura vengono accumulate in memoria dall'interprete e scaricate su disco solo alla chiusura. Dimenticare di chiudere un file può causare perdita di dati.

Il costrutto `with` risolve il problema automaticamente: chiude il file non appena si esce dal blocco indentato.

```
with open('dati/voti.txt', 'r', encoding='utf-8') as f:
    for riga in f:
        print(riga)
```

Il pattern da imparare è:

```
with open(percorso, modalità, encoding='utf-8') as nome_handler:
    for riga in nome_handler:
        # elabora la riga
```

Leggere un file CSV

Un formato comune è il **CSV** (Comma-Separated Values): ogni riga contiene campi separati da virgole. Esempio — `dati/voti.txt`:

```
Alice,Rossi,28
```

Marco,Bianchi,24
Sofia,Ferrari,30

Per leggere e stampare ogni riga in modo strutturato:

```
with open('dati/voti.txt', 'r', encoding='utf-8') as f:
    for riga in f:
        campi = riga.strip().split(',')
        nome = campi[0]
        cognome = campi[1]
        voto = campi[2]
        print(nome, cognome, '→ voto:', voto)
```

`strip()` rimuove il carattere di a capo (`\n`) che ogni riga porta con sé;
`split(',')` divide la riga in una lista di campi. L'output sarà:

Alice Rossi → voto: 28
Marco Bianchi → voto: 24
Sofia Ferrari → voto: 30

Questo approccio funziona con file di qualsiasi lunghezza: mille righe richiedono lo stesso codice di tre.

Scrivere su un file

Per scrivere si apre il file in modalità `'w'` e si usa il metodo `write()`:

```
with open('output.txt', 'w', encoding='utf-8') as f:
    f.write('prima riga\n')
    f.write('seconda riga\n')
```

`write()` non aggiunge automaticamente il carattere di a capo: va inserito manualmente con `\n`. La modalità `'w'` **sovrascrive** il file se esiste già.

Perché il file va chiuso: buffering Le operazioni di scrittura su disco sono costose: accedere alla memoria fisica richiede tempo. Per ottimizzare, Python non scrive ogni `write()` immediatamente — accumula le operazioni in una zona temporanea di memoria (il **buffer**) e le scarica su disco tutte in una volta alla chiusura del file. Se il file non viene chiuso, il contenuto del buffer potrebbe andare perso. Il costrutto `with` garantisce la chiusura automatica non appena si esce dal blocco, rendendo superfluo ricordarsi di chiamare `f.close()` a mano.

Metodi del file handler

Metodo	Cosa restituisce
<code>for riga in f</code>	Una riga alla volta (include <code>\n</code>)
<code>f.read()</code>	Tutto il contenuto come un'unica stringa
<code>f.readlines()</code>	Tutte le righe come lista di stringhe

Metodo	Cosa restituisce
<code>f.write(s)</code>	— (scrive la stringa <code>s</code> nel file)

La differenza tra i tre metodi di lettura è concreta. Dato un file `saluti.txt`:

```
ciao
mondo
```

- `for riga in f` — ad ogni iterazione `riga` vale `'ciao\n'`, poi `'mondo\n'`: si elabora una riga alla volta senza caricare tutto in memoria;
- `f.read()` — restituisce `'ciao\nmondo\n'`: un'unica stringa lunga con tutti i ritorni a capo dentro;
- `f.readlines()` — restituisce `['ciao\n', 'mondo\n']`: una lista dove ogni elemento è una riga.

Per la maggior parte degli usi, il ciclo `for riga in f` dentro un `with` è la soluzione più pratica: legge riga per riga, non occupa più memoria del necessario e si adatta a file di qualsiasi dimensione.

4. Git e GitHub per scaricare file di dati

Quando si vuole condividere una cartella con file di dati, un modo conveniente è usare **GitHub**: una piattaforma che ospita cartelle versionate tramite **Git**, un software che tiene traccia di tutte le modifiche ai file.

Per scaricare una cartella da GitHub sul proprio computer si usa il comando `git clone`:

```
git clone https://github.com/utente/nome-repository
```

Questo crea nella cartella corrente una copia locale della repository. Il vantaggio rispetto a uno zip è che se il proprietario aggiorna i file, basta eseguire `git pull` dalla cartella clonata per sincronizzare le modifiche senza ricaricare tutto.